Data Structures and Algorithm

COURSE CODE: CS-201

TOPIC: ARRAYS

COURSE INSTRUCTOR: HABIBA ARSHAD

Today’s Goal

- Operations on Array

CLO Covered in this Lecture

CLO2: **Apply** and design various data structures methods for specified applications.

Operations on Linear Array

We can perform various operations on a linear array:

- **Traversing-** processing each element of the array list.
- **Inserting-** adding new elements in the array list.
- **Deleting-** removing an element from the array list.
- **Sorting-** arranging the elements of the list in some sorting order.
- **Merging-** combining the elements of two array lists in a single array list.

Overflow & Underflow

Overflow: Attempt to insert an element into an array exceeding its size limit.

Underflow: Attempt to delete or traverse an element from an empty array.

Traversing Linear Arrays

Traversing is accessing and processing (visiting) each element of the data structure exactly ones

Traversing Linear Arrays

Traversing is accessing and processing (visiting) each element of the data structure exactly ones

Traversing Linear Arrays

Traversing is accessing and processing (visiting) each element of the data structure exactly ones

Traversing Linear Arrays

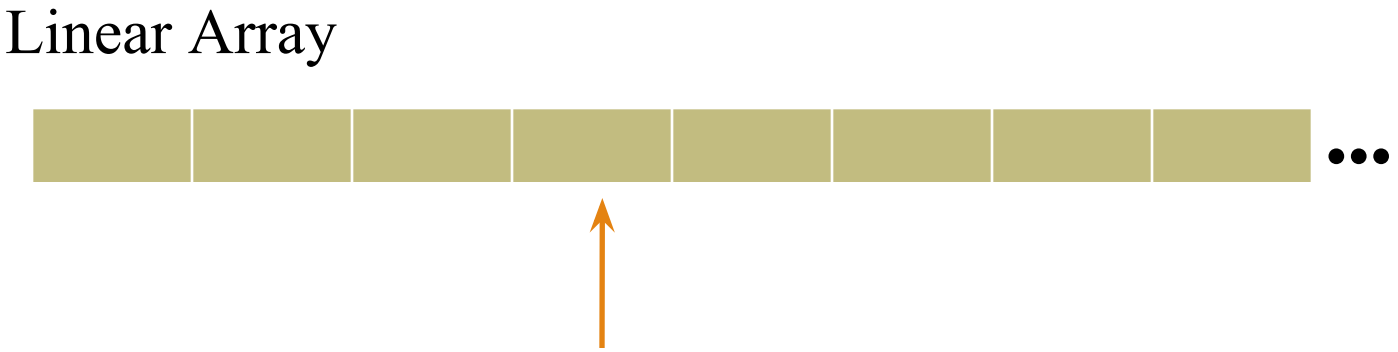
Traversing is accessing and processing (visiting) each element of the data structure exactly ones

Traversing Linear Arrays

Traversing is accessing and processing (visiting) each element of the data structure exactly ones

Traversing Linear Arrays

Traversing is accessing and processing (visiting) each element of the data structure exactly ones



Traversing Algorithm

We can process each element of an array with the help of an index set.

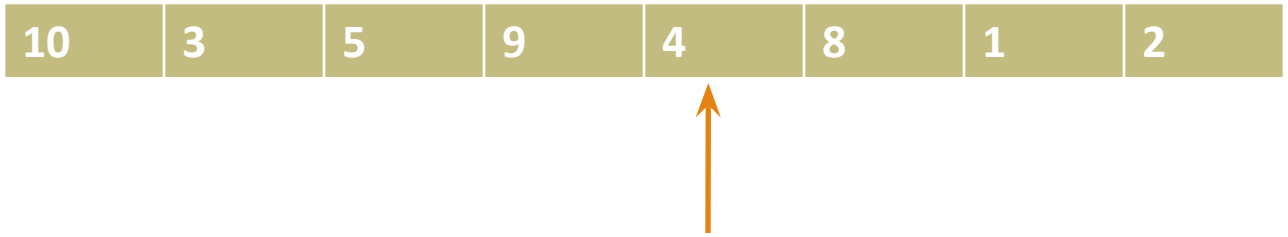
Consider a Linear Array(LA) list given with lower bound(LB) and upper bound(UB) that contains 'n' number of elements. We can traverse the list with the given algorithm.

1. [Initialize counter] Set K:= LB.
2. Repeat Steps 3 and 4 while K $\leq$ UB
3. [Visit element] Apply PROCESS to LA[K]
4. [Increase counter] Set K:= K + 1

   [End of Step 2 loop]
5. Exit.

Searching in Linear Arrays

Search ‘4’ in the following linear array:



Types of Searching in Arrays

- Linear Search
- Binary Search (can only be applied on sorted array)

Linear / Sequential Search

Sequential search is also known as linear or serial search. It follows the following step to search a value in array.

- Visit the first element of array and compare its value with required value.
- If the value of array matches with the desired value, the search is complete.
- If the value of array does not match, move to next element and repeat same process up till to the length of array until the desired element is found.

Linear Search Algorithm

1. Start
2. Set i = 0
3. Repeat steps 4 & 5 while i < UB
4. IF (item==LA[i]) THEN GOTO Step 6
5. Set i = i + 1
6. Print ‘record found at index i’
7. Print ‘item not found’
8. Stop

Time Complexity

Running time for Linear Search is O(n) as it depends on input size.

Binary Search

Binary search is a quicker method of searching for value in the array. But binary search is **applied only on sorted array**.

1. It locates the middle element of array and compare with desired number.
2. If they are equal, search is successful and the index of middle element is returned.
3. If they are not equal, it reduces the search to half of the array.
4. If the search number is less than the middle element, it searches the first half of array.

Otherwise it searches the second half of the array. The process continues until the required number is found or loop completes without successful search.

Binary Search Algorithm

```
 if UB== -1, then print 'underflow' & return.
Repeat while (LB<=UB)
{
 mid=(LB+UB)/2;
      if (LA[mid]==item)
      print 'item found';   return LA[mid];
     elseif (item<LA[mid])
           UB=mid-1;
      else
      LB=mid + 1;
}
        print 'item not in list';
```

Time Complexity

Running time for Binary Search is O(log n) as it divides the array in two half if required element is not found

Questions?